

BearLibTerminal / API reference

Separate pages covering different library parts in greater detail (*work in progress*):

- [Configuration](#)
- [Input](#)
- [Output](#)

The API consists of ~20 functions and a number of constants.

- Initialization and configuration: [open](#), [close](#), [set](#)
- Output state: [color](#), [bkcolor](#), [composition](#), [layer](#)
- Output: [clear](#), [clear area](#), [crop](#), [refresh](#), [put](#), [pick](#), [pick color](#), [pick bkcolor](#), [put ext](#), [print](#), [measure](#)
- Input: [state](#), [check](#), [has input](#), [read](#), [peek](#), [read str](#)
- Utility: [delay](#), [color from name](#), [color from argb](#)

Specifying the exact list of functions is tricky as underlying dynamic-link library exports an language-agnostic set of calls. The actual list of functions available to each programming language is slightly different.

For example, strings can be encoded using at least three character sizes: one (UTF-8 or ANSI), two (UTF-16) and four (UTF-32) bytes. Therefore for functions taking string arguments the dynamic-link library provides three separate calls. The C/C++ header file hides this fact by providing functions for more usual *char* and *wchar_t* string types, e. g. **print** and **wprint**. For some functions there are also variants with printf-like formatting, e. g. **printf** and **wprintf**. On the other hand, C# only use UTF-16 strings and formatting support is usually done by overloading the same function. So for C# there is only one printing function while C/C++ gets four.

Here the API is described briefly and mostly from pure C perspective. Language-specific peculiarities are listed on [separate pages](#) (*not yet done*).

open

```
int terminal_open();
```

This function initializes BearLibTerminal instance, configuring the window with default parameters: 80×25 cells, Fixedsys Excelsior font, white text on a black background. This function does not bring the window to screen. The window is not shown until the first call to [refresh](#). Note that unless the library is initialized with successful call to **open**, all other library functions will do nothing but return immediately with return code (if any) indicating an error.

The function returns boolean value where false means initialization has failed. Details may be found in the log file (named bearlibterminal.log by default).

close

```
void terminal_close();
```

Symmetric to [open](#), this function closes the window and deinitializes BearLibTerminal instance.

set

```
int terminal_set(const char* s); // + setf, wset, and wsetf flavours
```

This is probably the most complex function in BearLibTerminal API. Configuring library options and mechanics, managing fonts, tilesets and even configuration file is performed with it.

The function accepts a “configuration string” with various options described in it:

```
window.title='game';  
font: UbuntuMono-R.ttf, size=12;  
ini.settings.tile-size=16;
```

For an information about configuration string format, library options and overall function behavior, refer to the separate [configuration](#) page.

color

void terminal_color(color_t color);

This function sets the current foreground color which will be used by all output functions called later. The **color_t** type is a 32-bit unsigned integer describing color in BGRA (0xAARRGGBB) format. This numeric color representation can be constructed directly or by [color from argb](#) and [color from name](#) utility functions. For most languages including C++ there is also an overloaded version taking color name as a string (see [color from name](#) and language-specific [notes](#)).

The numeric value of current foreground color can be retrieved by reading *TK_COLOR* [state](#).

bkcolor

void terminal_bkcolor(color_t color);

Similar to [color](#) this function sets the current background color. Otherwise the function behaves exactly as its foreground counterpart. Note that only the first, lowest [layer](#) of cells has background.

The numeric value of current background color can be retrieved by reading *TK_BKCOLOR* [state](#).

composition

void terminal_composition(int mode);

This function sets character composition mode. When composition is off, putting a tile in a cell (by [put](#) or [print](#)) simply replaces the contents of that cell. When composition is on, however, the tile is added to the cell's tile stack. This has visual effect of combining several tiles into one. There is no enforced limit to number of tiles in a single cell. Note that each tile in the stack has its own foreground color and offset (see [put ext](#)).

Parameter **mode** may be one of: *TK_OFF* (default), *TK_ON*.

Current composition mode can be retrieved by reading *TK_COMPOSITION* [state](#).

layer

void terminal_layer(int layer);

This function selects the current layer of character cells. The argument is an index from 0 to 255 where 0 is the lowest (default) layer. Only the first layer has background, for layers 1 and above the background color set by [bkcolor](#) has no effect. Note that [clear area](#) affects only the current layer while [clear](#) wipes the entire scene.

Layers are useful for various reasons. One is that BearLibTerminal allows tiles bigger than one character cell. But the scene is drawn cell by cell in fixed left-to-right, top-to-bottom order. This makes impossible for the big tile to properly cover cells to the right and below because those will be drawn later. With layers the scene can be separated into several parts with strict Z-ordering between them.

Another use of layers is to logically separate the scene. Layers can be cleared and updated independently, so one can place parts of the scene in separate layers and update them without touching the others (for example, animated dungeon level and static UI).

Current layer index can be read from `TK_LAYER` [state](#) slot.

clear

```
void terminal_clear();
```

This function clears entire scene (all [layers](#)). It also sets background color of every cell to the currently selected background color.

clear_area

```
void terminal_clear_area(int x, int y, int w, int h);
```

This function clears a part of the currently selected [layer](#). The arguments specify top-left corner and a size of a rectangular area to be cleared. When called on the first layer, it also sets background color of affected cells to the currently selected background color.

crop

```
void terminal_crop(int x, int y, int w, int h);
```

This function sets a crop area of the current [layer](#). Dimensions of the area are expressed in cells. Cropping is disabled either by setting area's width or height to zero or by clearing entire scene with [clear](#).

refresh

```
void terminal_refresh();
```

This function commits the scene for output. It also has an effect of redrawing the scene.

BearLibTerminal does not draw to screen immediately upon [put](#) or [print](#) calls. Instead, the scene is constructed off-screen in a double-buffered manner. If the contents of the window are destroyed for some reason (for example, the window has been obstructed and operating system asking it to refresh), BearLibTerminal redraws an already committed “frontbuffer” scene. Only when this [refresh](#) function is called the modified scene will be actually brought to screen.

The first call to this function since library initialization will show the window on screen. Between the [open](#) and first [refresh](#) calls, the window stays invisible.

put

```
void terminal_put(int x, int y, int code);
```

This function puts a tile associated with the given character code into a cell with coordinates **x**, **y**. If the specified code is not associated with any tile, library use not-a-character tile with thin rectangle on it.

Note that character code must be an Unicode code point. Even if the currently selected font is a bitmap font with only a handful of tiles, BearLibTerminal internally maps those tiles to the proper Unicode code points. For example, euro sign if present will always be available by code point U+20AC unless the library is misconfigured.

pick

```
int terminal_pick(int x, int y, int index);
```

This function returns a code of a symbol/tile in the specified cell on a current [layer](#). The **index** parameter specifies an index of a tile in the cell. If there is no such tile in the cell (or the cell is empty), the function will return 0. Therefore to enumerate tiles in a cell, you should increment **index** until zero is returned.

The function does a reverse codepage translation of a tile code in accordance to *terminal.encoding* option.

pick_color

```
color_t terminal_pick_color(int x, int y, int index);
```

Returns a color of a tile in the specified cell.

pick_bkcolor

```
color_t terminal_pick_bkcolor(int x, int y);
```

Returns a background color of the specified cell.

put_ext

```
void terminal_put_ext(int x, int y, int dx, int dy, int code, color_t* corners);
```

This function is an advanced version of [put](#) allowing to also specify the tile offset and paint tile corners in different colors.

The **dx** and **dy** parameters specify the tile offset in pixels relative to the normal position of that tile in a cell. When using [composition](#) there may be more than one tile in a cell and each of them will have its own offset values. Note that offset does not change the drawing order in any way. Even if the tile is visually shifted to the right or below the cell it belongs to, it will be drawn in usual cell-by-cell order and may become covered by the other tiles. Use [layers](#) in that situation.

The **corners** parameter specifies an array of four **color_t** values describing individual colors of tile corners in the following order: top-left, bottom-left, bottom-right, top right (counter-clockwise starting from top-left). This feature allows smooth gradients and transitions between tiles. As with offset values, each tile in a cell has its own corner colors. If NULL is passed as the **corners** parameter, the current foreground color is used.

print

```
dimensions_t terminal_print(int x, int y, const char* s); // + printf, wprint and  
wprintf flavours
```

```
dimensions_t terminal_print_ext(int x, int y, int width, int height, int align,
const char* s); // + terminal_[w]print[f]_ext
```

This function prints a string starting from given coordinates. Each individual symbol is placed separately as if by calling [put\(_ext\)](#), i. e. **print** takes full advantage of selected [color](#), [layer](#) and [composition](#) options.

print_ext prints a string with optional auto-wrapping and alignment. Alignment may be one or a combination of: TK_ALIGN_DEFAULT (top-left), TK_ALIGN_LEFT, TK_ALIGN_RIGHT, TK_ALIGN_CENTER (center horizontally), TK_ALIGN_TOP, TK_ALIGN_BOTTOM, TK_ALIGN_MIDDLE (center vertically).

Behavior of **print** function may be modified by a number of tags in the string:

- Foreground and background colors: [color=red] or [bkcolor=gray]. Color names are parsed by [color from name](#) function.
- Arbitrary character codes: [U+E001] or [0xE001].
- Symbol combining: a[+]^ will produce something similar to â. Combining is done as if by placing two tiles into single cell with [composition](#) enabled.
- Pixel offset: [offset=4,8]. This adds an offset to every character placed by **print**, as if they were printed by [put_ext](#) function.
- Font change: [font=somename]. Enables output in [additional font faces](#).

Colors, pixel offset and font shift can be canceled by placing a symmetric closing tag, e. g. [/color] or [/offset]. Note that these are simple set/reset commands and they cannot be truly nested. There is no need to manually cancel every tag mentioned in a string because all of the effects are local to a single **print** call.

```
terminal_printf("[offset=%d,%d]g[+][color=red]^[/color] (red-hooded goblin)", dx,
dy);
```

To print a single [or] bracket, just double it.

This function returns the size of a string printed on the screen.

measure

```
dimensions_t terminal_measure(const char* s); // all [w]measure[f] flavours
dimensions_t terminal_measure_ext(int width, int height, const char* s); // +
terminal_[w]measure[f]_ext
```

This function returns the size a string would have if printed on the screen. **measure_ext** measures a string auto-wrapped within bbox (width, height). Note that unlike [print_ext](#) there is no alignment argument since it does not affect the size.

state

```
int terminal_state(int slot);
```

This function returns the current value of the state slot. These slots represent the library state: current foreground color, number of rows or columns, size of a cell, whether the key is pressed or not, etc.

Most of the states are updated only by reading input events with [read](#) function. These states are kept consistent to each other, e. g. this key combination check will work correctly even if the keys are long released:

```
int key = terminal_read();
```

```

if (key == TK_A)
{
    if (terminal_state(TK_SHIFT))
    {
        // Shift+A
    }
    else
    {
        // Just A
    }
}

```

The **state** function and input queue are described in greater detail on the [separate page](#) about input. There is also a list of all properties that can be retrieved with the **state** function on that page.

check

```
int terminal_check(int slot);
```

This is a simple wrapper around [state](#) function. In a lot of programming languages numerical and boolean types are not compatible so checking logically boolean properties with [state](#) may become unnecessary verbose. So you can use **check** to test some property.

The wrapper is “backported” to C/C++ [API](#) from the other bindings for better similarity between them.

has_input

```
int terminal_has_input();
```

This function returns a boolean value indicating input availability: *true* means that next [read](#) call will return a value without blocking.

read

```
int terminal_read();
```

This function returns the next event from input queue. If there is no more events in the queue, **read** will wait for an event to come. If this blocking behaviour is not desirable, you can check [has_input](#) before calling **read**.

peek

```
int terminal_peek();
```

Similar to [read](#) but does not remove the event from the internal queue. The next **peek** or **read** call will return the same value. This is non-blocking function: if the queue is empty, it will not wait and return 0.

read_str

```
color_t terminal_read_str(int x, int y, char* buffer, int max); // + read_wstr
wide-char overload
```

This function does a simple blocking read of a string without any parsing. User input is displayed at specified coordinates and also limited to specified length so it can be used in game interface. The function displays user input within current layer and restores the scene before return. It is an application's responsibility to keep the result on screen.

The function returns the size of a string if user has confirmed input by pressing Enter, or `TK_INPUT_CANCELLED` if operation was cancelled by pressing ESCAPE or closing the window.

If `read_str` does not fit your needs, see [TK CHAR/TK WCHAR](#) states allowing for manual text input.

delay

```
void terminal_delay(int period);
```

Suspend the program execution for a specified amount of time (in milliseconds).

color_from_name

```
color_t color_from_name(const char* name); // + color_from_wname version for
wchar_t string
```

This function returns a numeric representation of a color from palette. The name is spelled in "[brightness]hue" form, e. g. "red", "light green", "darker #905025".

Possible values for brightness are: lightest, lighter, light, dark, darker, darkest.

Hue (base color) may be specified in a number of ways:

- By name: grey (or gray), red, flame, orange, amber, yellow, lime, chartreuse, green, sea, turquoise, cyan, sky, azure, blue, han, violet, purple, fuchsia, magenta, pink, crimson, transparent.
- In a hexadecimal format: #RRGGBB or #AARRGGBB, e. g. #80905025
- In a comma-separated decimal format: R,G,B or A,R,G,B e. g. 128,200,150,75
- As a plain integer number formatted to a string, e. g. 16744448

New named colors may be added to the default palette in a call to [set](#):

```
terminal_set("palette.octarine = #50FF25");
```

Or in a [configuration file](#):

```
[Palette]
lush = dark 80,255,37
```

Note that [print](#) uses this function to parse the values of its color formatting tags.

color_from_argb

```
color_t color_from_argb(uint8_t alpha, uint8_t red, uint8_t green, uint8_t blue);
```

This function is a simple utility function combining four separate 8-bit color channels into one 32-bit value.

Show pagesource

Old revisions

Media Manager

Login

[Sitemap](#)

[Back to top](#)